

# 递推

递推是一种重要的数学方法，比如当我们需要求第n个位置的状态的时候，需要先计算出第n个状态前的某些状态。

**比较典型的例子就是斐波拉契数列。**

$$f[1]=1;$$

$$f[2]=1;$$

$$f[n]=f[n-1]+f[n-2] \quad (n \geq 3)$$

如果我们需要知道斐波拉契数列中的第k项的值，我们就需要要先知道k-1项和k-2项的值，我们要知道k-1项的值就必须知道k-2和k-3项的值。而且该数列只有递推公式而没有通项公式

# 台阶问题

小明在走爬楼梯，他一次可以走一步，也可以走两步，请问他如果要达到第n层台阶，她有多少种走法。因为最终的结果可能特别大，所以我们让结果对10000007取余。  
输出最终的结果

输入：

4

输出：

5

输入：

6

输出：

13

$n \leq 10000$

# 台阶问题

## 第一种办法：找规律(打表)

|      |                     |   |
|------|---------------------|---|
| 第一层: | 1                   | 1 |
| 第二层: | 11;2                | 2 |
| 第三层: | 111;12;21           | 3 |
| 第四层: | 1111;112;121;211;22 | 5 |
| 第五层  | ...                 | 8 |

多找几层，我们就会发现规律，第k层的方案数等于前两层方案数之和。

想一想：如果每次可以走一步，两步，三步,递推公式又是什么呢？

# 台阶问题



先开一个一维数组 $f[10010]$ ,  $f[i]$ 表示走到第 $i$ 层台阶的方案数

当小明走到第 $k$ 层台阶的时候, 由于一次可以走一步, 也可以走两步, 那么小明上一次的位置要么是在 $k-1$ 处, 要么是在 $k-2$ 处, 只有这两种可能。

**所以:**

到达台阶 $k$ 的方案数=到达台阶 $k-1$ 的方案数+到达台阶 $k-2$ 的方案数

即:  **$f[k] = f[k-1] + f[k-2]$**

# 台阶问题 (升级版)

有N级的台阶，你一开始在底部，每次可以向上迈最多K级台阶（最少1级），问到达第N级台阶有多少种不同方式。由于答案可能很大，你需要输出mod 100003后的结果。

输入：  
5 2

输出：  
8

前面我们计算步数的时候，如果一次可以走两步，我们是直接算出走一步，两步的方案数，同样走三步也是如此，但是此题是走k步，而且k有可能比较大，直接笔算出来不太现实。所以我们需要重新思考一下

$N \leq 100000, K \leq 100。$

# 台阶问题 (升级版)

我们可以分两种情况讨论：

第一种情况： $m > k$ 的时候



$$f[m] = f[m-1] + f[m-2] + \dots + f[m-k];$$

写成代码形式是：

```
for (int i=1; i<=k; i++)  
    f[m] += f[m-i];
```

# 台阶问题 (升级版)

第二种情况:  $m \leq k$  的时候



我们先假设  $k=5$ , 我们计算下 1-5 层台阶的方案数, 看下有什么规律

第一步: 1  
第二步: 2  
第三步: 4  
第四步: 8  
第五步: 16

# 台阶问题 (升级版)

我们也可以用数学方法推导：



当 $m < k$ 的时候，第 $m$ 层台阶可以由 $m-1, m-2 \dots 1$ 层台阶到达，同时由于一次可以走 $k$ 步，所以我们可以直接从第0层台阶到第 $m$ 层台阶，所以应该加上第零层的方案数。 $f[m] = f[m-1] + f[m-2] + \dots + f[1] + f[0]$ ;

写成代码形式：

```
for(int i=1; i<=m; i++)  
    f[m] += f[m-i];
```



# 台阶问题 (升级版)

也就是说:

当 $m \leq k$ 的时候:

```
for(int i=1; i<=m; i++)  
    f[m] += f[m-i];
```

当 $m > k$ 的时候

```
for(int i=1; i<=k; i++)  
    f[m] += f[m-i];
```

我们可以写在一起变为:

```
for(int i=1; i<m&& i>=k; i++)  
    f[m] += f[m-i];
```

终止条件是k和m中较小的那一个

# 台阶问题(再升级)

简单的走台阶问题我们已经会了，还是每次可以走一步，可以走两步。我们这次改变一下策略，如果台阶中某些台阶坏掉了是不可以走的，那请问走到台阶n有多少种方案。输入n，再输入n个数，分别表示该层台阶的状态，0表示可以走，1表示不可以走。求方案数，仍然是让结果对10000003取余

输入：

6

0 0 1 0 0 0

输出：

4

$n \leq 10000$

# 台阶问题(再升级)

同样的，第k层台阶只能由第k-1层或者k-2层台阶到达，但是如果某一层台阶是坏的，那么就不能由那个位置到达第k层台阶，同样的，如果第k层台阶本来就是坏的，那么没有任何一种办法走到第k层台阶。

如果第k层台阶是坏的：

$$f[k]=0$$

如果第k层台阶是好的：

{

    如果第k-1层台阶是坏的

$$f[k]=f[k-2];$$

    如果第k-2层台阶是坏的

$$f[k]=f[k-1];$$

    如果都是坏的

$$f[k]=0;$$

    如果都是好的

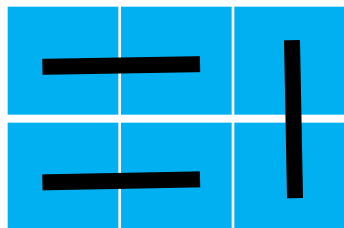
$$f[k]=f[k-1]+f[k-2];$$

}

可以简化为： $f[k]=f[k-1]+f[k-2]$

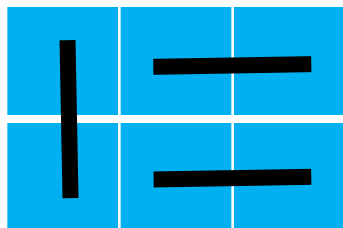
# 骨牌覆盖1

有一个 $2*N$ 的长方形方格，用 $1*2$ 的骨牌去铺满方格，  
请问有多少种铺法。



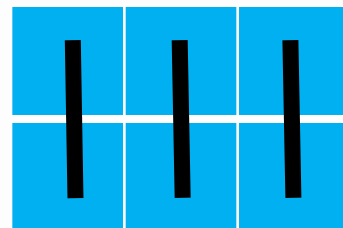
输入：  
3

输出：  
3



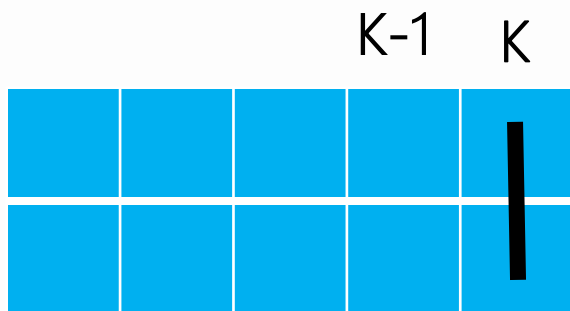
输入：  
5

输出：  
8

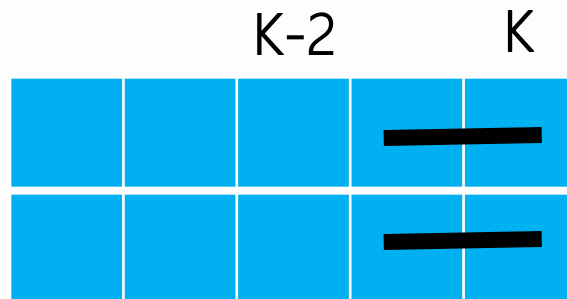


$N \leq 40$

# 骨牌覆盖1



如果第k个格子是竖着放的  
 $f[k]=f[k-1]$

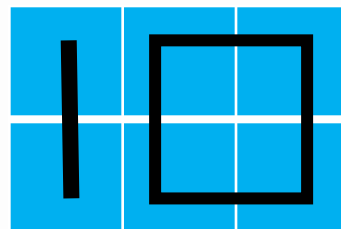
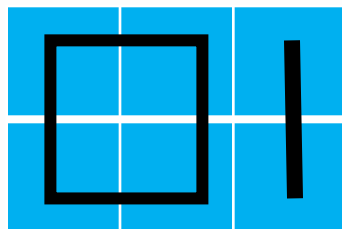
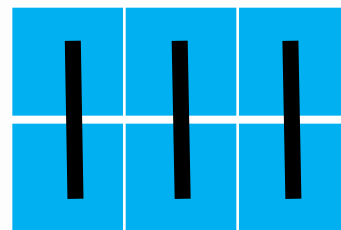
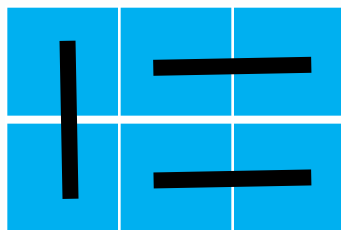
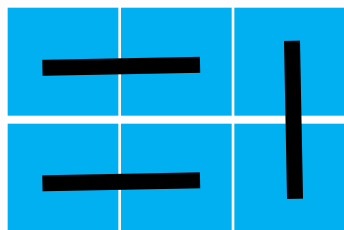


如果第k个格子是横着放的  
 $f[k]=f[k-2]$

合起来就是  $f[k]=f[k-1]+f[k-2]$

# 骨牌覆盖2

有一个 $2 \times N$ 的长方形方格，有 $1 \times 2$ 和 $2 \times 2$ 的骨牌去铺满方格，请问有多少种铺法。



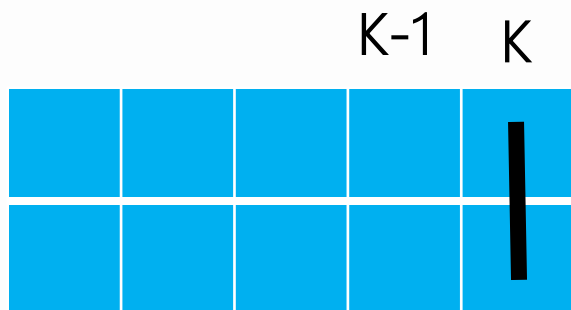
输入：

3

输出：

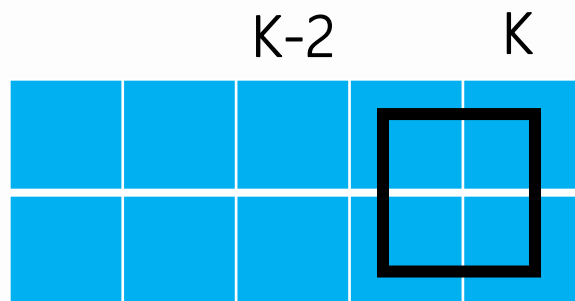
5

# 骨牌覆盖 2



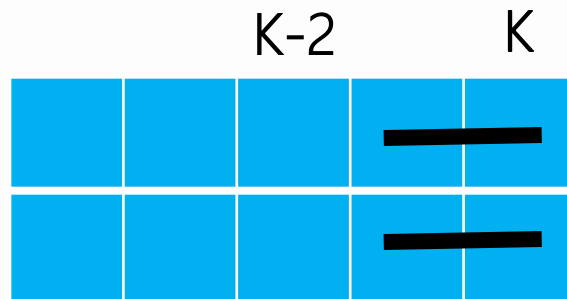
如果第k个格子是竖着放的

$$f[k]=f[k-1]$$



如果第k个格子是放的2\*2的

$$f[k]=f[k-2]$$



如果第k个格子是横着放的

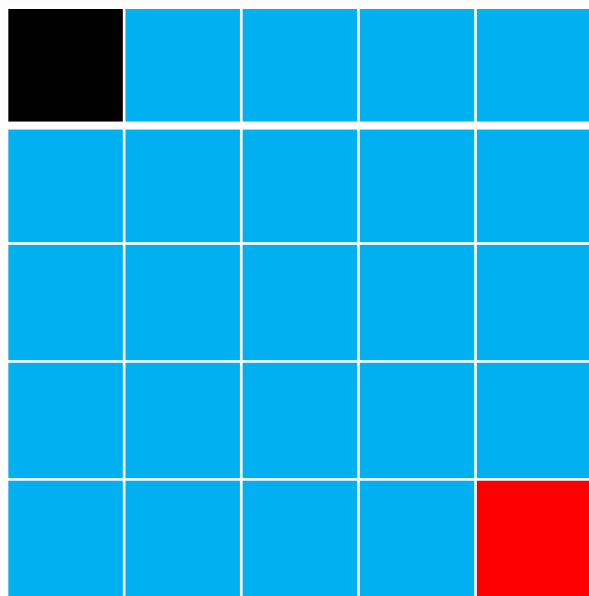
$$f[k]=f[k-2]$$

合起来就是：

$$f[k]=f[k-1]+2*f[k-2]$$

# 走方格

已知一个 $n \times n$ 的方格，小明在方格的左上角  $(1,1)$ ，要走到方格的右下角  $(n,n)$ ，请问有多少种走法，已知小明只可以向右和向下移动。由于走法可能特别多，所以结果对123456取余



输入：  
2

输出：  
2

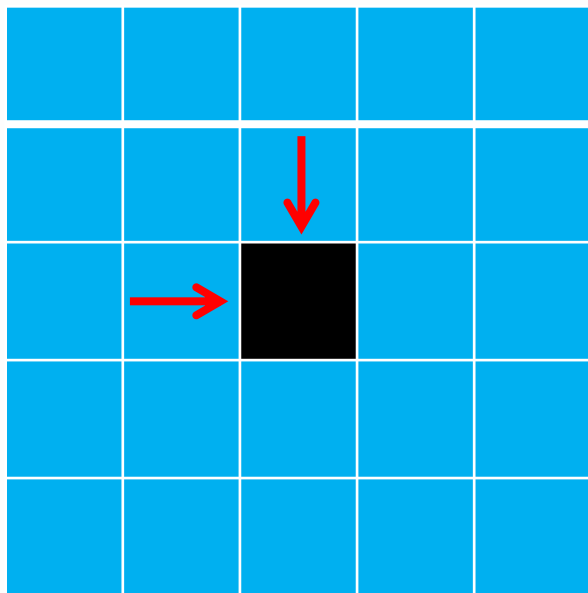
输入：  
3

输出：  
6

$n \leq 100$



# 走方格



对于表格中的每一个位置  $(i,j)$  ,由于小明只可以往右或者往下走, 所以 $(i,j)$ 只能从 $(i-1,j)$ 和 $(i,j-1)$ 这两个位置走过来。

设走到点 $(i,j)$ 的方案数为 $f[i][j]$

$$f[i][j]=f[i-1][j]+f[i][j-1]$$

但是有一些特殊的点：比如第一行的点，第一列的点，只能由他左边的点或者上面的点到达。

```
for(int i=1;i<=n;i++)  
for(int j=1;j<=n;j++)
```

```
f[i][1]=1;  
f[1][j]=1;
```

# 走方格

```
int n;  
scanf("%d", &n);  
for(int i=1; i<=n; i++)  
f[i][1]=1; //初始化第一列方案数为1  
for(int j=1; j<=n; j++)  
f[1][j]=1; //初始化第一行方案数为1  
for(int i=2; i<=n; i++)  
{  
    for(int j=2; j<=n; j++)  
    {  
        f[i][j]=(f[i-1][j]+f[i][j-1])%123456;  
    }  
}  
printf("%d", f[n][n]);
```

# 走方格(升级版)

已知一个 $n*m$ 的方格，小明在方格的左上角  $(1,1)$  ,要走到方格的右下角  $(n,n)$  , 请问有多少种走法，已知小明只可以向右和向下移动,并且中间有些方格是塌陷的(1表示不可以走，0表示可以走)，结果对123456取余。

输入

2 3

0 1 0

0 0 0

输出：

1

$n,m \leq 10$

0

# 数字三角形

写一个程序来查找从最高点到底部任意处结束的路径，使路径经过数字的和最大。每一步可以走到左下方的点也可以到达右下方的点。

```

    7
   3 8
  8 1 0
 2 7 4 4
4 5 2 6 5
```

最大路径和为30:

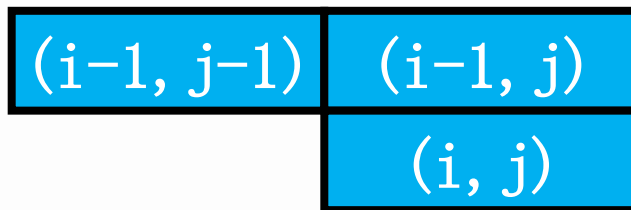
路径是：7--3--8--7--5

如果按照贪心策略，从上往下取每次取最大是7--8--1--7--5，结果为28，不是最优解，从下往上取每次取最大是6--4--1--8--7，结果是26，也不是最优解。那我们应该如何找最大值呢？

# 数字三角形

7  
3 8  
8 1 0  
2 7 4 4  
4 5 2 6 5

对于三角形中的每一个位置  $(i, j)$ ，  
他只能由  $(i-1, j)$  和  $(i-1, j-1)$  这  
两个位置到达



如果需要到达  $(i, j)$  这个位置，就需要我们选择从  $(i-1, j-1)$  过来或者从  $(i-1, j)$  到达。比如如果我要到达5这个位置，我就需要选择从2或者从7到达，为了保证路径最大，肯定是选择从7到达5。

设到达  $(i, j)$  位置的点的最大路径是  $f[i][j]$ ，那么：

$$f[i][j] = \max(f[i-1][j], f[i-1][j-1]) + a[i][j];$$

# 数字三角形

我们先求出从第1层到第2层的最长路径。

7

3 8

8 1 0

2 7 4 4

4 5 2 6 5

7

10 15

再求出第2层到第3层的最长路径。

10 15

23 16 15

再求出第3层到第4层的最长路径。

23 16 15

25 23 20 19

最后找出最大值就是30

再求出第3层到第4层的最长路径。

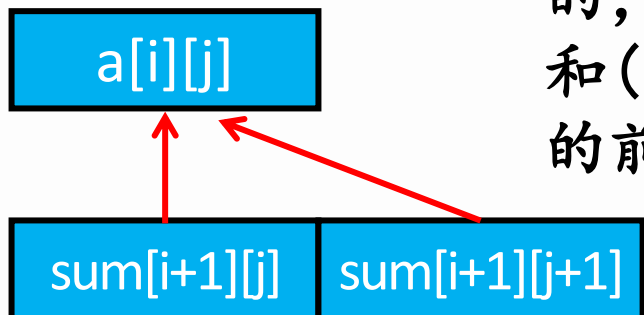
25 23 20 19

29 30 25 26 24

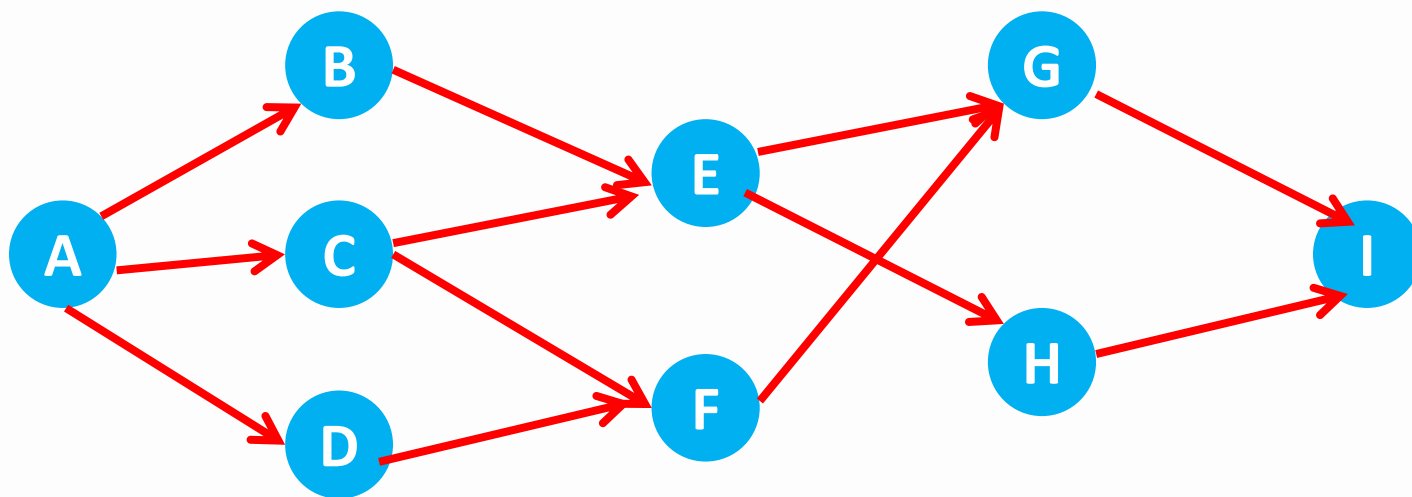
# 数字三角形

```
for(int j=1;j<=n;j++)//从最后一行开始往上找当前层的最大路径
sum[n][j]=a[n][j];//最后一行的最大路径就是他自身
for(int i=n-1;i>=1;i--)
{
    for(int j=1;j<=i;j++)
    {
        sum[i][j]=max(sum[i+1][j],sum[i+1][j+1])+a[i][j];
    }
}
```

要保证到达  $(i, j)$  这个位置的路径是最大的，并且  $(i, j)$  这个位置只能通过  $(i+1, j)$  和  $(i+1, j+1)$  到达，所以应该选择到达他的前一个位置最大的那一条路径



# 最短路径





# 踩方格

有一个方格矩阵，矩阵边界在无穷远处。我们做如下假设：

- a. 每走一步时，只能从当前方格移动一格，走到某个相邻的方格上；
- b. 走过的格子立即塌陷无法再走第二次；
- c. 只能向北、东、西三个方向走；

请问：如果允许在方格矩阵上走 $n$ 步，共有多少种不同的方案。2种走法只要有一步不一样，即被认为是不同的方案。

输入：

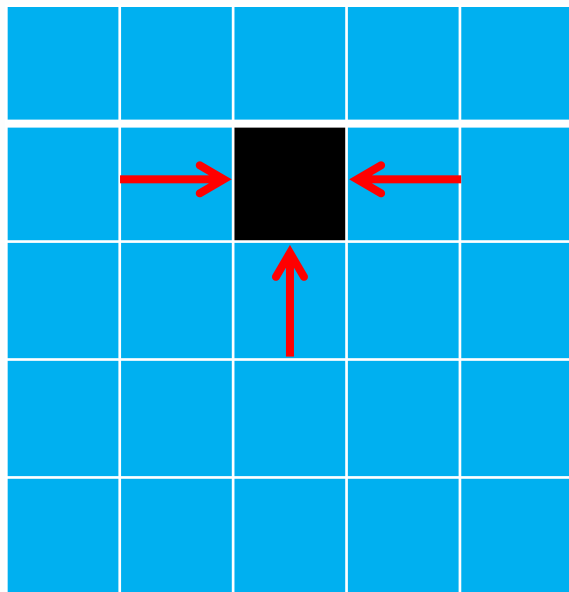
2

输出：

7

$n \leq 20$

# 踩方格



方格中走到  $(i, j)$  位置的方案数等于走到方格中  $(i, j-1)$ ,  $(i, j+1)$ ,  $(i-1, j)$  的方案数之和。

$$f[i][j] = f[i-1][j] + f[i][j-1] + f[i][j+1];$$

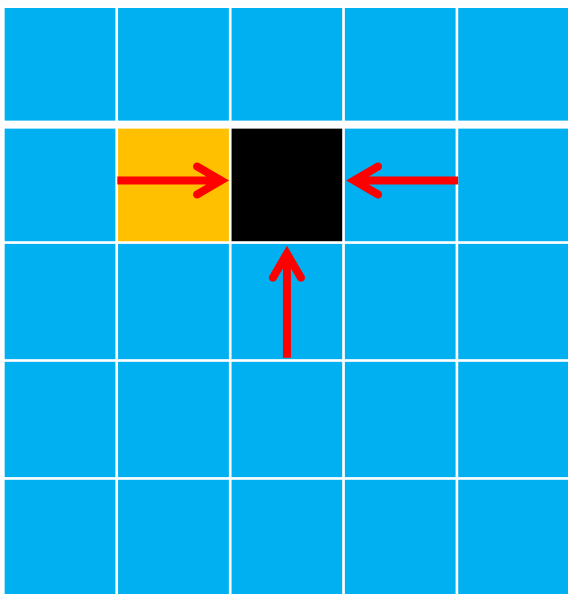


**错误一：**按照递推的规则，比如我们按照行列都从1开始推，在我们求  $f[i][j]$  位置的方案数之前，我们只知道  $f[i-1][j]$  和  $f[i][j-1]$  的方案数，而不知道  $f[i][j+1]$  的方案数。

# 踩方格

错误二：  $f[i][j] = f[i-1][j] + f[i][j-1] + f[i][j+1]$ ;

陷阱：只能向左，上，右移动，并且移动过的方格马上就会塌陷，



当上一步到达  $(i, j-1)$  的时候，他可以从  $(i, j)$  过来，即从右边过来，但是如果他从右边过来，那么下一步就不能再往左边走了，因为该方格已经塌陷了。即并不是走到  $(i, j-1)$ ,  $(i, j+1)$  的方案都可以走到走到  $(i, j)$ 。所以递推公式错误

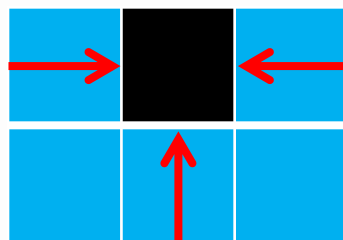
# 踩方格

所以我们分情况讨论：

设走 $k$ 步，最后一次向右走的方案数为 $r[k]$ ；

设走 $k$ 步，最后一次向左走的方案数为 $l[k]$ ；

设走 $k$ 步，最后一次向上走的方案数为 $t[k]$ ；



我们可以得到递推公式为：

$r[k] = t[k-1] + r[k-1]$ ；//上一步不可以向左走到达

$l[k] = t[k-1] + l[k-1]$ ；//上一步不可以向右走到达

$t[k] = t[k-1] + r[k-1] + l[k-1]$ ；//所有方案都可以

初始化：

$r[1] = 1$ ；

$l[1] = 1$ ；

$t[1] = 1$ ；

最终： $f[k] = r[k] + l[k] + t[k]$ ；

# 踩方格

我们可以把上面的等式化简：

$r[k] = t[k-1] + r[k-1]$ ; // 上一步不可以向左走到达

$l[k] = t[k-1] + l[k-1]$ ; // 上一步不可以向右走到达

$t[k] = t[k-1] + r[k-1] + l[k-1]$ ; // 所有方案都可以

$$r[k] + l[k] + t[k] = 2 * (t[k-1] + r[k-1] + t[k-1]) + t[k-1];$$

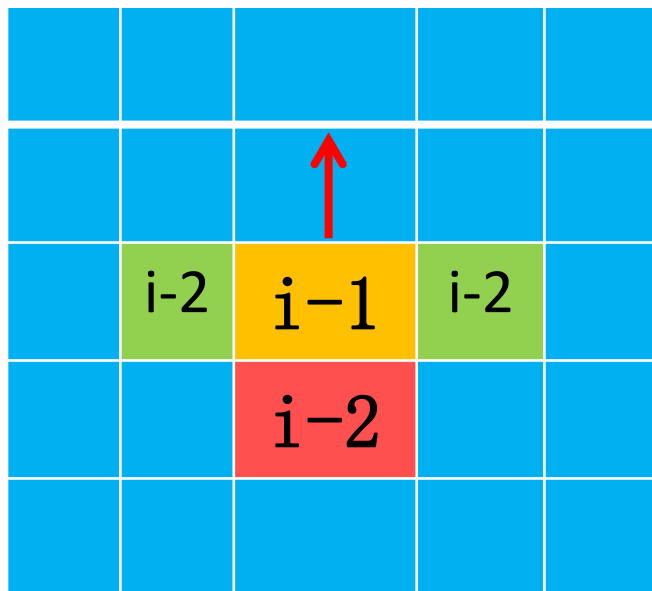
$$f[k] = 2 * f[k-1] + t[k-1];$$

$$t[k-1] = t[k-2] + r[k-2] + l[k-2] = f[k-2];$$

最终得到：  $f[k] = 2 * f[k-1] + f[k-2]$

初始值：  $f[0] = 1$ ;      $f[1] = 3$ ;

# 踩方格



当第 $i-1$ 步走到表格中黄色的区域的位置的时候，方案数为 $f[i-1]$ ；

第一种情况：无论是从哪个方向到达该位置，如果下一次（第 $i$ 步）向上（北）走，那么走 $i$ 步的方案数是 $f[i-1]$

第二种情况：如果上一次是从下方（红色）往上（北）走到达黄色位置，说明下一步（第 $k$ 步）左边和右边都可以走，往上走的情况，第一种情况已经考虑了就不在重复计算了，方案数是 $f[i-2]*2$ 。

第三种情况，如果上一次是从左边或者右边（绿色）到达黄色方格，说明下一次只可以往右

或者左走（方向不变，向上已经考虑了。），到达黄色方格的所有方案数是 $f[i-1]$ ，再减去从下方到达黄色方格的方案数 $f[i-2]$ ，即 $f[i-1] - f[i-2]$

# 递推总结

做这一类的题的时候，首先我要确定他是需要递推求解的。大概题干意思是：经过 $k$ 次变换到达状态 $m$ ，求方案数(结果)，最主要的是当前状态是可以用前一个已经确定的状态表达的，

做递推的题的时候，一定不能死板，要懂的灵活变通，要分析实际问题，找到当前状态和上一状态的联系，一般用一个多维数组来表示，数组的维度根据实际情况来定，但是要注意一点：这个状态是唯一确定的，并且分析问题一定要全面，找出所有到达该状态的上一状态的可能性。

如果做这一类型的题有困难的同学，也无需担忧，先仔细分析题干，再多练习题，多次练习之后，做该类题就会得心应手了。